

TestbenchGen_Report

Enhanced LLM-Aided Testbench Generation — LLM4ChipDesign homework · Anuj Apte (NYU Tandon, MS Computer Engineering)

1. Summary

I ran the course pipeline (description + RTL → initial testbench → Python golden model → patterns with golden outputs → self-checking final testbench) on two tutorial examples and on one module of my own, and verified every final testbench with Icarus Verilog 12. I then injected bugs into my own module and showed that the generated testbench fails on them and passes after the fix.

Run	Kind	Patterns	Checks (pass / fail)	Result
mux2to1 (tutorial 1)	combinational	8	8 / 0	PASS
adder4bit (tutorial 2)	arithmetic	32	64 / 0	PASS
sat_adder8 (custom)	8-bit saturating adder + cin	40	80 / 0	PASS
sat_adder8 with injected bug	bug detection	40	70 / 10	FAILS as intended

Tutorial examples run: **2-to-1 multiplexer** and **4-bit adder**. Custom module: **sat_adder8**. All artifacts, the executed notebook (testbenchgen_hw.ipynb) and logs are in the repository folder HW_TestbenchGen/.

2. LLM setup (please read)

Who played the LLM. The notebook is the course's own pipeline: three LLM calls per module (testbench + patterns, golden model, testbench updater), with the course's prompts unchanged. In live mode it calls gpt-4o through the OpenAI API (the tutorial default; the course code uses temperature 0.7 for the testbench and updater calls and 0.2 for the golden model, with max_tokens 4000/3000/8000).

The environment in which I prepared this submission had no OpenAI access, so the runs reported here used a replay mode: each LLM response was written by Claude (Anthropic, model claude-sonnet-5-5) for the exact prompt the pipeline sends, and is stored in llm_responses/<module>/. Claude chose the test patterns (directed corner cases plus seeded random values) and wrote each golden model from the natural-language spec only. The repetitive per-test check blocks in the final testbench were emitted by a small helper (llm_responses/build_responses.py) that copies expected values from the golden model, to avoid transcription errors. Everything after the LLM call — response parsing, running the golden model, writing the artifacts, and all iverilog/vvp simulations — is real, live execution in the notebook. Pasting an OpenAI key into the single marked cell switches the same notebook to live GPT-4o runs (generated artifacts would then differ from those submitted).

Parameters / prompt choices. I changed no course prompts or sampling parameters. The edits to the course code are three small additions to LLMClient (optional openai import, client created only in live mode, and the replay provider). The prompt choice that mattered most was the specification text for the custom module: it spells out ports/widths, the exact saturation rule, named corner cases (total = 255 vs 256, carry-in alone causing saturation) and an explicit request for at least 20 patterns mixing directed corners and random values. The pipeline's own format requirements (binary-string patterns, TESTBENCH_CODE:/TEST_PATTERNS: markers, all $\$display$ in the initial block) shaped the responses.

3. Runs: DUT interfaces and commands

Module	Ports (direction, width)	Compile and simulate (run from HW_TestbenchGen/)
mux2to1	in a[1], b[1], sel[1]; out y[1]	iverilog -g2012 -o mux/sim.vvp mux/mux2to1.v mux/testbench_final.v vvp mux/sim.vvp
adder4bit	in a[4], b[4]; out sum[4], carry[1]	iverilog -g2012 -o adder/sim.vvp adder/adder4bit.v adder/testbench_final.v vvp adder/sim.vvp
sat_adder8	in a[8], b[8], cin[1]; out sum[8], sat[1]	iverilog -g2012 -o custom/sim.vvp custom/sat_adder8.v custom/testbench_final.v vvp custom/sim.vvp

Each run produced the four required artifacts in its folder: testbench_initial.v, golden_model.py, test_patterns_with_golden.json, testbench_final.v (plus the DUT and simulation_output.log). All three final testbenches compile with no warnings and end with ALL TESTS PASSED:

```

mux2to1      : Total Checks: 8   Passed: 8   Failed: 0
adder4bit    : Total Checks: 64  Passed: 64  Failed: 0
sat_adder8   : Total Checks: 80  Passed: 80  Failed: 0   (one check per output per pattern)

```

4. Part II — understanding the artifacts (4-bit adder)

golden_model.py. It defines adder4bit_golden(a, b): one parameter per DUT input, named like the ports, receiving unsigned integers (the pipeline converts the LLM's binary strings with `int(x, 2)`). It returns a dict keyed by the DUT output names, {'sum': ..., 'carry': ...}. The reference is computed from the spec, not the RTL: add the operands at full Python precision (no wraparound), then `sum = total & 0xF` and `carry = 1 if total > 15 else 0`.

test_patterns_with_golden.json. A list with one record per pattern. Each record has one key per DUT input holding the stimulus as a decimal integer, and an expected_outputs object holding the golden model's integer result for every DUT output. For example: {"a": 15, "b": 15, "expected_outputs": {"sum": 14, "carry": 1}}. It is produced by running the golden model over every pattern and is what the updater reads to place expected constants in the testbench.

Updater effect (before → after). testbench_initial.v (142 lines) only applies stimuli and prints them with `$display`; it can never fail. testbench_final.v (635 lines) adds, for every test, a #10 settle delay, an `===` compare of each output against its golden constant with ✓/X messages showing actual and expected values, and pass/fail counters, followed by a summary and ALL TESTS PASSED / SOME TESTS FAILED. Case equality means an x/z output is counted as a failure. Added for Test 1 of the adder:

```

    $display("Test 1: a=%b b=%b", a, b);
+   #10; // let outputs settle
+   if (sum === 4'd0) begin
+       $display(" ✓ Test 1: sum = %0d (expected: 0)", sum);
+       passed_tests = passed_tests + 1;
+   end else begin
+       $display(" ✗ Test 1: sum = %0d (expected: 0)", sum);
+       failed_tests = failed_tests + 1;
+   end

```

5. Part III — custom module sat_adder8

Design choice and spec. An 8-bit unsigned saturating adder with carry-in: `total = a + b + cin` at full precision; if `total ≤ 255` then `sum = total`, `sat = 0`, otherwise `sum = 255`, `sat = 1`. It has a sharp boundary (total exactly 255 is not saturation; 256 is the first saturating case) and the carry-in can be the only thing that pushes the result over, which makes it a good target for corner-case testing. The full natural-language description is in the notebook (Section 6).

Quality of tests. The generated pattern file has **40 patterns** (80 output checks): 24 directed corner cases and 16 seeded random patterns. Coverage: 20 saturating and 20 non-saturating cases; 10 patterns with total exactly 255 (must not saturate); 7 with total exactly 256 (first saturating case); 5 where the carry-in alone decides saturation. Excerpt of the checking logic in custom/testbench_final.v:

```
// Test 5 (corner)
a = 8'b11111111; b = 8'b00000000; cin = 1'b1;
$display("Test 5: a=%b b=%b cin=%b", a, b, cin);
#10; // let outputs settle
if (sum === 8'd255) begin
    $display(" ✓ Test 5: sum = %0d (expected: 255)", sum);
    passed_tests = passed_tests + 1;
end else begin
    $display(" x Test 5: sum = %0d (expected: 255)", sum);
    failed_tests = failed_tests + 1;
end
end
if (sat === 1'd1) begin
    $display(" ✓ Test 5: sat = %0d (expected: 1)", sat);
    passed_tests = passed_tests + 1;
end else begin
    $display(" x Test 5: sat = %0d (expected: 1)", sat);
    failed_tests = failed_tests + 1;
end
end

// Test 6 (corner)
```

Final run: **Passed: 80, Failed: 0 — ALL TESTS PASSED** (custom/simulation_output.log).

6. Part IV — bug detection

Bug injected. In sat_adder8.v the flag assign sat = full[8]; was changed to assign sat = (full >= 9'd255); — an off-by-one that wrongly flags saturation when the total is exactly 255 while the sum value stays correct. Only 511 of the 131,072 input combinations ($\approx 0.4\%$) trigger it, so directed boundary patterns are what catch it.

Failing run (iverilog -g2012 -o custom/bug_demo/buggy.vvp custom/bug_demo/sat_adder8_buggy.v custom/testbench_final.v; vvp custom/bug_demo/buggy.vvp):

```
Test 3: a=11111111 b=00000000 cin=0
  x Test 3: sat = 1 (expected: 0)
Test 9: a=11111110 b=00000001 cin=0
  x Test 9: sat = 1 (expected: 0)
...
===== Test Summary =====
Total Checks: 80
Passed: 70
Failed: 10
SOME TESTS FAILED
=====
```

10 of 80 checks fail, all of them on directed boundary patterns, none on the random ones. **Fix and passing confirmation** (the buggy line restored to assign sat = full[8];, same testbench):

```
===== Test Summary =====
Total Checks: 80
Passed: 80
Failed: 0
ALL TESTS PASSED
=====
```

Extra: mutation sweep. Four more plausible bugs were also run against the same final testbench; none survived:

Mutant	Failed checks (of 80)	Verdict
M1: sat = (full >= 255) (off-by-one, the demo bug)	10	detected
M2: saturates to 8'hFE instead of 8'hFF	20	detected
M3: carry-in ignored	13	detected
M4: sat flag ignores cin (sum still correct)	5	detected
M5: operand b replaced by a	28	detected

7. Reproducibility and caveats

Open `testbenchgen_hw.ipynb` and run all cells; it installs `openai/icarus Verilog` if missing, needs no key in replay mode, and regenerates every artifact (the shipped notebook is the executed version, `Icarus Verilog 12.0`, Linux). No API key or credential is stored anywhere in the repository. Caveats: (1) the LLM outputs are Claude-authored recordings, not GPT-4o output; (2) a live GPT-4o run would produce different, and possibly imperfect, artifacts — which is exactly what the simulation step is there to check; (3) the Part II prose in the notebook describes the recorded run.